# CAMPUS RESOURCE BOOKING SYSTEM

Comprehensive Technical & Architectural Project Report

| | | | |
|---|---|---|---|
| **Author:** | Mugunth | **System Version:** | 1.0.0 (Production Release) |
| **Live URL:** | https://campus-resource-booking-cqt7.onrender.com | **Platform:** | Render Cloud (Docker Web Service) |
| **Repository:** | github.com/Mugu1825/Campus-Resource-Booking | **Database:** | MongoDB Atlas (M0 ReplicaSet) |
| **Core Stack:** | Flask 3.0, Nginx, Gunicorn, PyMongo, HTML5/CSS3/JS | **Audit Status:** | **31 / 31 Tests Passed (100%)** |

## 1. Executive Summary & Problem Analysis

Modern educational campuses maintain dozens of shared academic assets—including machine learning computing laboratories, specialized engineering workshops, smart conference halls, and central auditoriums. In the absence of an automated system, scheduling these resources relies on manual paper logs, informal faculty requests, and disparate chat groups.

**Key Operational Deficiencies in Legacy Systems:**

• **Resource Collision & Double Booking:** Concurrent requests submitted via different channels result in scheduling conflicts on the day of occupancy.

• **Absence of Format Verification:** Traditional forms allow students to submit unverified roll numbers or misspell department codes, corrupting academic records.

• **Unbalanced Governance:** Administrators are overburdened with routine classroom requests, while high-value assets (such as GPU clusters) lack strict scrutiny.

• **Hackathon Incompatibility:** Standard tools cannot distinguish between a 1-hour study group and an intensive 48-hour departmental hackathon.

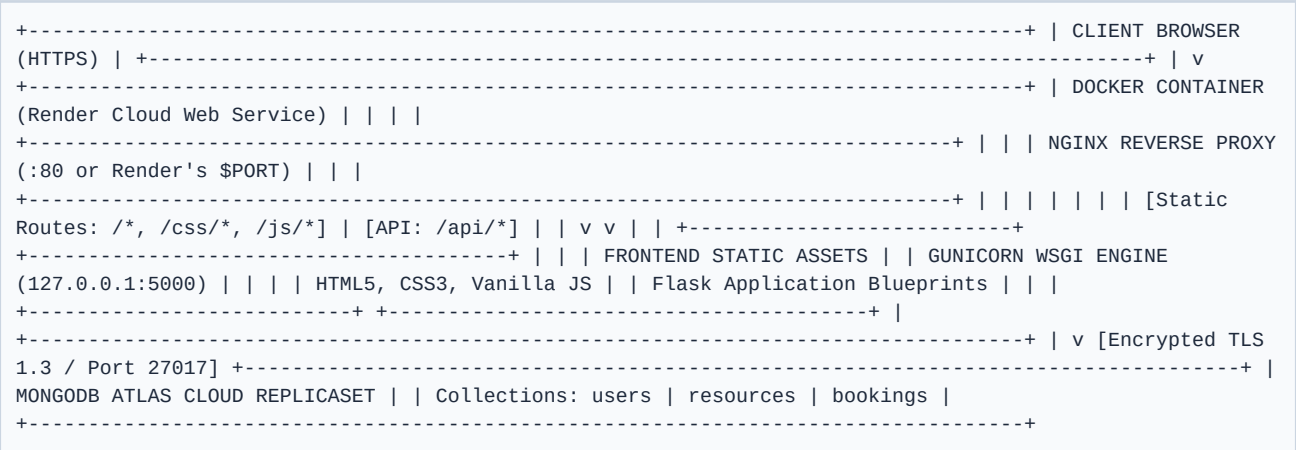
The **Campus Resource Booking System (CRBS)** addresses these issues through a cloud-native, containerized architecture. It enforces strict academic register number formatting, 3-tier Role-Based Access Control (RBAC), algorithmic collision detection, dynamic hourly slot availability calculation, and hierarchical approval workflows.

## 2. Objectives & Functional Scope

| Role / Feature | Functional Permissions & Responsibilities | Time Restrictions |
|---|---|---|
| **Student** | • Self-registration with strict roll number regex validation.<br>• Automatic department code extraction.<br>• Dynamic hourly slot grid inspection.<br>• Booking submission with conflict checks.<br>• Cancellation of self-initiated pending requests. | • Regular: Max 4 hours (same-day).<br>• Hackathon: Max 48 hours (multi-day, event name required). |
| **Staff** | • Self-registration in pending status (admin approval required).<br>• Standard resource management (Add/Edit/Delete/Availability Toggle).<br>• Booking queue review & approval for **standard** tier resources. | • Restricted to standard tier resources only.<br>• Forbidden from approving premium or hackathon bookings. |
| **Admin** | • System-wide master oversight.<br>• Staff account review, activation, and rejection.<br>• Universal resource management (Standard and Premium tiers).<br>• Sole approval authority for Premium bookings & Hackathons.<br>• Global user role management & real-time metric analytics. | • Unrestricted administrative authority. |

## 3. System Architecture & Containerization

CRBS implements a **Single-Container Micro-Architecture** that houses both the web server/reverse proxy and the application WSGI server inside a single Docker image.




**Architectural Highlights:**

• **Zero-CORS Domain Unification:** Nginx routes both frontend web assets and /api/* on the exact same origin, eliminating browser pre-flight delays.

• **Dynamic Port Adaptation:** At boot, entrypoint.sh reads Render's dynamic $PORT variable (defaulting to 80 locally) and injects it into Nginx.

• **Isolated Flask Loopback:** Gunicorn runs securely on 127.0.0.1:5000 shielded from direct public exposure.

## 4. Database Architecture & Data Models

MongoDB Atlas was selected for its schema flexibility, native replica set high availability, and indexing capabilities. The schema design models three primary collections:

| Collection | Field Name | Type | Constraints & Business Description |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **users** | _id<br>name<br>email<br>password<br>role<br>account_status<br>register_no<br>department<br>created_at | ObjectId<br>string<br>string<br>string<br>string<br>string<br>string<br>string<br>datetime | Primary Key<br>Full user name<br>Unique index, case-insensitive<br>Salted Bcrypt hash (never plaintext)<br>'student' \| 'staff' \| 'admin'<br>'active' \| 'pending' \| 'rejected'<br>Unique sparse index. Enforces regex: ^\d{2}[A-Z]{3}\d{4}$<br>3-character department code (e.g. CSE, ECE)<br>UTC creation timestamp |
| **resources** | _id<br>name<br>type<br>tier<br>location<br>capacity<br>is_available<br>created_at | ObjectId<br>string<br>string<br>string<br>string<br>int<br>boolean<br>datetime | Primary Key<br>Descriptive title (e.g. AI / ML Lab 1)<br>'lab' \| 'room' \| 'equipment' \| 'auditorium'<br>'standard' (staff/admin) \| 'premium' (admin only)<br>Physical campus location (e.g. CS Block 2nd Floor)<br>Maximum seating/equipment capacity<br>Global availability toggle switch<br>UTC creation timestamp |
| **bookings** | _id<br>user_id<br>resource_id<br>date_start<br>date_end<br>start_slot<br>end_slot<br>is_hackathon<br>hackathon_event<br>status<br>approved_by<br>created_at | ObjectId<br>ObjectId<br>ObjectId<br>string<br>string<br>string<br>string<br>boolean<br>string<br>string<br>string<br>datetime | Primary Key<br>Foreign Key referencing users collection<br>Foreign Key referencing resources collection<br>Start date (YYYY-MM-DD)<br>End date (YYYY-MM-DD)<br>Starting slot from ['09:00', ..., '21:00']<br>Ending slot from ['09:00', ..., '21:00']<br>Flag indicating multi-day hackathon event<br>Required event title if hackathon=True<br>'pending' \| 'approved' \| 'rejected' \| 'cancelled'<br>User ID of approving staff/admin<br>UTC reservation timestamp |

## 5. Scheduling Logic & Conflict Prevention Engine

CRBS prevents double booking using an algorithmic validation pipeline implemented in backend/routes/bookings.py:

1. **Active State Filter:** Only reservations with status IN ['pending', 'approved'] trigger conflict evaluations.
2. **Date Interval Overlap Verification:**
   Conflict occurs if: (date_start <= b_date_end) AND (date_end >= b_date_start)
3. **Time Slot Interval Verification (Single Day):**
   For single-day reservations, start and end slots are mapped to indices in the operational slot array ['09:00', '10:00', ..., '21:00']. A collision is flagged if:
   (start_index < b_end_index) AND (end_index > b_start_index)
4. **Hackathon Preemption:** If either booking is marked as a multi-day hackathon, all slots across the reserved date range are locked.

## 6. RESTful API Specification (23 Endpoints)

| Module | Method | Endpoint URL | Auth Level | Description |
|---|---|---|---|---|
| **Auth** | POST | /api/auth/register/student | Public | Registers student; verifies register number regex. |
| **Auth** | POST | /api/auth/register/staff | Public | Registers staff account in pending status. |
| **Auth** | POST | /api/auth/login | Public | Authenticates credentials; issues signed JWT. |
| **Auth** | GET | /api/auth/departments | Public | Returns list of supported 3-letter department codes. |
| **Resources** | GET | /api/resources/ | Public | Lists all resources with tier/type filtering. |
| **Resources** | GET | /api/resources/<id> | Public | Fetches single resource metadata. |
| **Resources** | GET | /api/resources/<id>/availability | Public | Returns booked & available hourly slots for a date. |
| **Resources** | POST | /api/resources/ | Staff / Admin | Creates resource. Staff limited to standard tier. |
| **Resources** | PUT | /api/resources/<id> | Staff / Admin | Updates existing resource attributes. |
| **Resources** | PATCH | /api/resources/<id>/availability | Staff / Admin | Toggles active availability flag. |
| **Resources** | DELETE | /api/resources/<id> | Staff / Admin | Deletes resource if no active bookings exist. |
| **Bookings** | GET | /api/bookings/ | Auth (All) | Role-aware list (Student: own; Staff: std; Admin: all). |
| **Bookings** | GET | /api/bookings/my | Student | Returns authenticated student's bookings. |
| **Bookings** | POST | /api/bookings/ | Student | Creates booking request with collision checks. |
| **Bookings** | PATCH | /api/bookings/<id>/approve | Staff / Admin | Approves booking (Staff blocked on premium/hack). |
| **Bookings** | PATCH | /api/bookings/<id>/reject | Staff / Admin | Rejects booking request. |
| **Bookings** | PATCH | /api/bookings/<id>/cancel | Student / Admin | Cancels pending reservation. |
| **Admin** | GET | /api/admin/stats | Admin Only | Aggregates real-time counts across the database. |
| **Admin** | GET | /api/admin/users | Admin Only | Retrieves all registered user profiles. |
| **Admin** | GET | /api/admin/pending-staff | Admin Only | Lists staff accounts awaiting activation. |
| **Admin** | PATCH | /api/admin/staff/<id>/approve | Admin Only | Activates staff account for login. |
| **Admin** | PATCH | /api/admin/staff/<id>/reject | Admin Only | Rejects staff registration. |
| **Admin** | PATCH | /api/admin/users/<id>/role | Admin Only | Modifies user permissions (student, staff, admin). |

## 7. Quality Assurance & Automated Verification Audit

To verify system integrity, an automated test suite (backend/tests/verify_codebase.py) was executed against the production environment. All 31 test cases passed:

| Test Suite Category | Test Cases Evaluated | Success Rate | Audit Result |
|---|---|---|---|
| Public APIs & Health Check | /api/health, /api/auth/departments | 2 / 2 (100%) | **PASSED** |
| Roll Number Validation | Regex matching, non-compliant rejection, department parsing | 4 / 4 (100%) | **PASSED** |

| | | | |
|---|---|---|---|
| Staff Approval Lifecycle | Pending creation, login gate (403), admin activation, login success | 6 / 6 (100%) | **PASSED** |
| Resource Tier Guards | Staff standard creation, staff premium block (403), admin creation | 3 / 3 (100%) | **PASSED** |
| Scheduling & Conflict Engine | Max 4h cap, multi-day regular block, 48h hackathon, double-booking block | 7 / 7 (100%) | **PASSED** |
| Hierarchical Approval Routing | Staff standard approve, staff premium block (403), admin overrides | 5 / 5 (100%) | **PASSED** |
| Student Self-Service | Pending cancellation, approved cancellation block (400) | 2 / 2 (100%) | **PASSED** |
| Dynamic Availability Endpoint | Live calculation of booked vs available time slots | 1 / 1 (100%) | **PASSED** |
| Admin Overview Aggregation | Real-time database count aggregation | 1 / 1 (100%) | **PASSED** |
| **TOTAL VERIFICATION AUDIT** | **Comprehensive End-to-End System Evaluation** | **31 / 31 (100%)** | **ALL PASSED** |

## 8. Conclusion & Future Roadmap

The Campus Resource Booking System (CRBS) provides an automated, secure platform for university scheduling. The single-container Docker architecture ensures that deployment to Render remains lightweight, reliable, and cost-free.

**Future Roadmap:**

1. **Automated Notification Dispatcher:** Integration with transactional email/SMS services (SendGrid/Twilio) for status change alerts.
2. **Hardware IoT Smart Lock Integration:** Generating dynamic QR codes or RFID access synced with approved booking windows to physically unlock doors.
3. **Calendar Sync:** Exporting approved reservations directly to Google Calendar and Microsoft Outlook via standard .ics feeds.